

JENKINS CI/CD PROJECT

Deploying a React (Vite) Application to Amazon EKS
using a Fully Automated CI/CD Pipeline

Project Report

Submitted by: PRITAM PRIYANSHU PATRA

SIC : 23BECD84

Branch: Electronics & Communication Engineering]

Institution: Silicon University, Bhubaneswar

Domain: DevOps

August 2026

Table of Contents

1. Project Overview	3
2. System Architecture	4
3. Application Development	5
4. Docker Configuration.....	6
5. Kubernetes Fundamentals — Local Verification	8
6. Jenkins Server Setup	10
7. Amazon EKS Cluster Setup	12
8. Jenkins Configuration for CI/CD	14
9. CI/CD Pipeline Workflow.....	16
10. Continuous Deployment	19
11. Kubernetes Deployment on Amazon EKS	21
12. Application Live Verification	23
13. Challenges Faced	24
14. Learning Outcomes	25
15. Conclusion	26

1. Project Overview

1.1 Objective

This project implements a complete, end-to-end CI/CD pipeline using Jenkins to automatically build, containerize, and deploy a React (Vite) frontend application onto an Amazon Elastic Kubernetes Service (EKS) cluster. The pipeline covers infrastructure provisioning, Jenkins configuration, source control integration, Docker image creation, Kubernetes deployment, and fully automated continuous deployment triggered directly by Git commits.

1.2 Problem Statement

A software development team wants to automate the deployment of their frontend application so that every code change pushed to Git automatically triggers a pipeline that builds the application, creates a Docker image, pushes it to a container registry, and deploys the latest version to an Amazon EKS cluster — without any manual intervention. This report documents the design, implementation, and verification of that pipeline.

1.3 Services and Tools Used

Category	Tool / Service
Compute	Amazon EC2 (Jenkins host, EKS worker nodes)
CI/CD	Jenkins (Declarative Pipeline)
Container Orchestration	Amazon EKS (Kubernetes)
Containerization	Docker, multi-stage build with Nginx
Container Registry	Amazon Elastic Container Registry (ECR)
Source Control	Git / GitHub
Identity & Access	AWS IAM (users, roles, aws-auth mapping)
CLI Tooling	AWS CLI, kubectl, eksctl
Frontend Framework	React + Vite

1.4 High-Level Project Workflow

The solution follows a linear, automated flow: a developer commits code to the Git repository; Jenkins detects the change and triggers a pipeline; the pipeline checks out the source, installs dependencies, builds the React application, builds a Docker image, and pushes it to Amazon ECR with a unique tag; Jenkins then updates the Kubernetes Deployment on the EKS cluster, and the updated application becomes available to end users through a Kubernetes Service backed by an AWS Load Balancer.

2. System Architecture

The architecture diagram below illustrates the full path from source-code commit to a running, publicly accessible application. Each stage is fully automated and triggered by the previous stage's success.

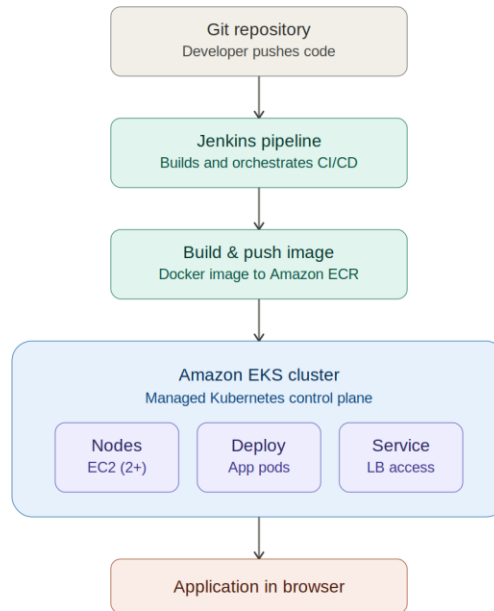


Figure 1: End-to-end CI/CD architecture — Git to Amazon EKS

2.1 Architecture Explanation

- **Git Repository:** the single source of truth for both application code and the Jenkins pipeline definition (Jenkinsfile). Every push to the main branch is the trigger for the entire pipeline.
- **Jenkins Pipeline (EC2):** orchestrates the build — checking out source code, installing dependencies, compiling the production build, and driving the Docker and Kubernetes stages.
- **Docker:** packages the compiled React application into a lightweight, portable container image using a multi-stage build.
- **Amazon ECR:** stores every built image under a unique, traceable tag (the Jenkins build number), giving a full history of deployable artifacts.
- **Amazon EKS Cluster:** the managed Kubernetes control plane and worker node group that runs the application containers.
- **Kubernetes Deployment:** maintains the desired number of application replicas and performs rolling updates whenever a new image is pushed.
- **Kubernetes Service (LoadBalancer):** exposes the Deployment externally via an AWS Elastic Load Balancer, giving end users a stable URL.

3. Application Development

The frontend was scaffolded using Vite's official React template (npm create vite@latest), which provides a fast development server and an optimized production build process. No additional application logic was required — the focus of this project is the delivery pipeline rather than the application itself.

3.1 Setup and Verification

- Node.js and npm installed and verified on the development machine.
- Project scaffolded with the default Vite + React template.
- Local development server and production build (npm run build) verified successfully.
- Project initialized with Git and pushed to a GitHub repository.

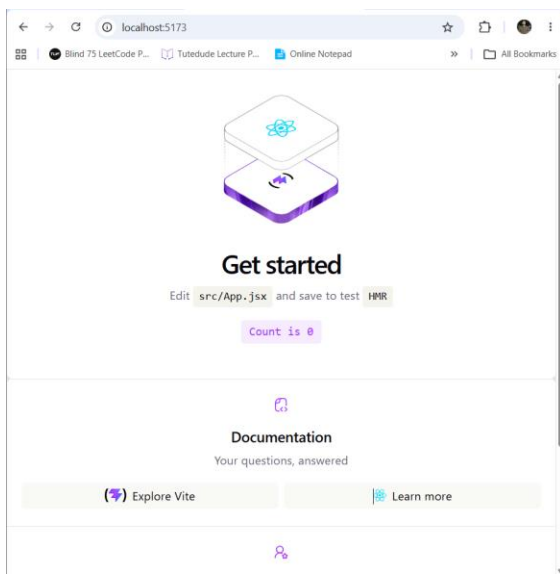


Figure 2: Default Vite + React application running locally

```
ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app$ ls -la
total 88
drwxrwxrwx 1 ptkun ptkun 4096 Aug 1 12:40 .
drwxrwxrwx 1 ptkun ptkun 4096 Aug 1 12:23 ..
-rwxrwxrwx 1 ptkun ptkun 253 Aug 1 12:37 .gitignore
-rwxrwxrwx 1 ptkun ptkun 1027 Aug 1 12:39 README.md
drwxrwxrwx 1 ptkun ptkun 4096 Aug 1 12:27 node_modules
-rwxrwxrwx 1 ptkun ptkun 568 Aug 1 12:39 eslint.config.js
-rwxrwxrwx 1 ptkun ptkun 373 Aug 1 12:39 index.html
drwxrwxrwx 1 ptkun ptkun 4096 Aug 1 12:48 public
-rwxrwxrwx 1 ptkun ptkun 83587 Aug 1 12:40 package-lock.json
-rwxrwxrwx 1 ptkun ptkun 617 Aug 1 12:39 package.json
drwxrwxrwx 1 ptkun ptkun 4096 Aug 1 12:39 src
drwxrwxrwx 1 ptkun ptkun 4096 Aug 1 12:27 test
drwxrwxrwx 1 ptkun ptkun 4096 Aug 1 12:39 types
-rwxrwxrwx 1 ptkun ptkun 161 Aug 1 12:37 vite.config.js
ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app$
```

Figure 3: React project folder structure

```
ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app$ npm run build
> jenkins-eks-react-app@0.0.0 build
> vite build

vite v5.0.8 building for production...
computing gzip size...
dist/asset-1234567890.js 0.47 kB | gzip: 0.30 kB
dist/asset-1234567890.css 2.30 kB | gzip: 1.30 kB
dist/asset-1234567890.html 190.00 kB | gzip: 101.00 kB

vite v5.0.8 building for production...
computing gzip size...
dist/asset-1234567890.js 0.47 kB | gzip: 0.30 kB
dist/asset-1234567890.css 2.30 kB | gzip: 1.30 kB
dist/asset-1234567890.html 190.00 kB | gzip: 101.00 kB

2 directories, 9 files
ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app$
```

Figure 4: Production build completed successfully (npm run build)

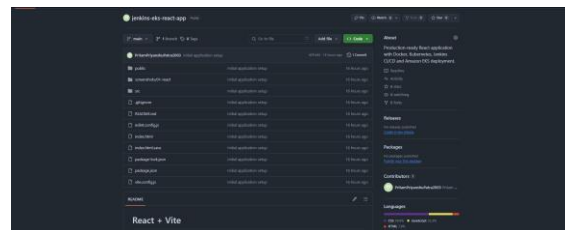


Figure 5: Project pushed to the GitHub repository

4. Docker Configuration

The application is packaged using a two-stage Dockerfile. The first stage installs dependencies and builds the production bundle using a Node.js base image; the second stage copies only the compiled static output into a minimal Nginx image, keeping the final runtime image small and free of build-time dependencies.

4.1 Dockerfile

```
# Stage 1: Build React application
#-----
FROM node:24 AS builder

WORKDIR /app

COPY . .

RUN npm install

RUN npm run build

# Stage 2: Serve using Nginx
#-----
FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

4.2 .dockerignore

```
node_modules
.git
dist
README.md
.gitignore
```

4.3 Build and Runtime Verification

The image was built locally and run as a standalone container to confirm the application served correctly through Nginx before being wired into the Kubernetes and Jenkins stages.

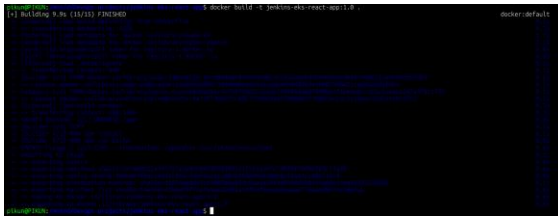


Figure 6: Docker image built successfully



Figure 7: Container running locally

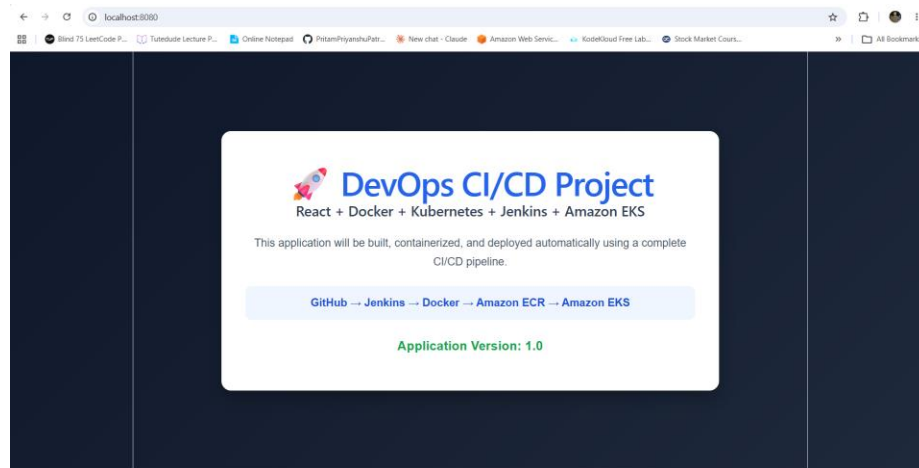


Figure 8: React application served from the Docker container, verified in browser

5. Kubernetes Fundamentals — Local Verification

Before deploying to the managed EKS cluster, core Kubernetes objects were exercised locally to validate the manifests and confirm correct behaviour: Pods, Deployments, Services, ConfigMaps, Secrets, and Namespaces, along with self-healing and rolling-update behaviour.

5.1 Pod Definition

```
apiVersion: v1
kind: Pod
metadata:
  name: react-pod
spec:
  containers:
  - name: react-container
    image: jenkins-eks-react-app:1.0
    ports:
    - containerPort: 80
```

5.2 Deployment, Service, and Self-Healing

```
plkum@IKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$ kubectl get deployments
kubectl get replicaset
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
react-deployment  3/3     3            3           3m31s
NAME      DESIRED   CURRENT   READY   AGE
react-deployment-64f9dbff55  3         3         3       3m31s
NAME      READY   STATUS    RESTARTS   AGE
react-deployment-64f9dbff55-7cd8  1/1     Running   0          3m31s
react-deployment-64f9dbff55-qwmdg  1/1     Running   0          3m31s
react-deployment-64f9dbff55-xc98g  1/1     Running   0          3m31s
plkum@IKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$
```

Figure 9: Deployment running with desired replica count

```
plkum@IKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$ kubectl get deployments
kubectl get replicaset
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
react-deployment  3/3     3            3           6m20s
NAME      DESIRED   CURRENT   READY   AGE
react-deployment-64f9dbff55  3         3         3       6m20s
NAME      READY   STATUS    RESTARTS   AGE
react-deployment-64f9dbff55-brpkw  1/1     Running   0          102s
react-deployment-64f9dbff55-qwmdg  1/1     Running   0          6m20s
react-deployment-64f9dbff55-xc98g  1/1     Running   0          6m20s
plkum@IKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$
```

Figure 10: ReplicaSet automatically replacing a terminated pod

```
plkum@IKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$ kubectl apply -f service.yaml
service/react-service created
plkum@IKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$ kubectl get services
NAME      TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes  ClusterIP  10.96.0.1    <none>        443/TCP   4h44m
react-service  NodePort  10.111.167.75 <none>        80:30652/TCP  61s
plkum@IKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$
```

Figure 11: Kubernetes Service created and exposing the Deployment

```
plkum@IKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$ minikube service react-service
NAME      NAME      TARGET PORT  URL
default  react-service  80           http://192.168.49.2:30652
Starting tunnel for service react-service.
NAME      NAME      TARGET PORT  URL
default  react-service  80           http://127.0.0.1:32961
Opening service default/react-service in default browser...
! http://127.0.0.1:32961
! Because you are using a Docker driver on linux, the terminal needs to be open to run it.
```

Figure 12: Application accessible via the Kubernetes Service

5.3 Rolling Updates, ConfigMaps and Secrets

A version-2 image was built and rolled out to confirm zero-downtime updates, and ConfigMaps / Secrets were used to demonstrate externalized configuration and sensitive-value management, independent of the container image.

```
apiVersion: v1
kind: ConfigMap
metadata:
```



```

name: react-config
data:
  APP_NAME: "React Demo"
  ENVIRONMENT: "Development"
  API_URL: "https://api.example.com"
  APP_VERSION: "2.1"

```

```

apiVersion: v1
kind: Secret
metadata:
  name: react-secret
type: Opaque
stringData:
  DB_USERNAME: admin
  DB_PASSWORD: password123

```

```

pkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$ kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
react-deployment-84c6c496cd-fnhh6   1/1     Running   0           9m2s
react-deployment-84c6c496cd-smwxj   1/1     Running   0           9m
react-deployment-b9fdd759c-qrs12    0/1     Pending   0           0s
react-deployment-b9fdd759c-qrs12    0/1     Pending   0           0s
react-deployment-b9fdd759c-qrs12    0/1     ContainerCreating   0           1s
react-deployment-b9fdd759c-qrs12    1/1     Running   0           1s
react-deployment-84c6c496cd-fnhh6   1/1     Terminating   0           9m57s
react-deployment-b9fdd759c-xcjsd     0/1     Pending   0           0s
react-deployment-84c6c496cd-fnhh6   1/1     Terminating   0           9m57s
react-deployment-b9fdd759c-xcjsd     0/1     Pending   0           0s
react-deployment-84c6c496cd-fnhh6   0/1     ContainerCreating   0           9m58s
react-deployment-b9fdd759c-xcjsd     1/1     Running   0           1s
react-deployment-84c6c496cd-fnhh6   0/1     Completed   0           9m58s
react-deployment-84c6c496cd-fnhh6   0/1     Completed   0           9m58s
react-deployment-84c6c496cd-hng8k   1/1     Terminating   0           9m59s
react-deployment-84c6c496cd-hng8k   1/1     Terminating   0           10m
react-deployment-b9fdd759c-pqgpt     0/1     Pending   0           0s
react-deployment-b9fdd759c-pqgpt     0/1     Pending   0           0s
react-deployment-b9fdd759c-pqgpt     0/1     ContainerCreating   0           0s
react-deployment-84c6c496cd-hng8k   0/1     Completed   0           10m
react-deployment-b9fdd759c-pqgpt     1/1     Running   0           0s
react-deployment-84c6c496cd-hng8k   0/1     Completed   0           10m
react-deployment-84c6c496cd-hng8k   0/1     Completed   0           10m
react-deployment-84c6c496cd-smwxj   1/1     Terminating   0           9m58s
react-deployment-84c6c496cd-smwxj   0/1     Completed   0           9m58s
react-deployment-84c6c496cd-smwxj   0/1     Completed   0           9m58s
react-deployment-84c6c496cd-smwxj   0/1     Completed   0           9m58s

```

Figure 13: Rolling update to a new image version

```

pkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$ kubectl describe configmap react-config
Name:         react-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
API_URL:
----
https://api.example.com

APP_NAME:
----
React Demo

ENVIRONMENT:
----
Development

BinaryData
=====
Events: <none>
pkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$

```

Figure 14: ConfigMap created for application configuration

```

pkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$ kubectl get secrets
NAME          TYPE      DATA   AGE
react-secret  Opaque    2       21s

```

Figure 15: Kubernetes Secret created

```

pkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$ kubectl create namespace dev
namespace/dev created
pkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$ kubectl get namespaces
NAME                STATUS   AGE
default             Active   26h
dev                 Active   7s
kube-node-lease     Active   26h
kube-public         Active   26h
kube-system         Active   26h
pkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/k8s$

```

Figure 16: Namespace isolation verified

6. Jenkins Server Setup

Jenkins was hosted on a dedicated Amazon EC2 instance. Java, Git, Docker, the AWS CLI, and kubectl were installed alongside Jenkins, and the Jenkins service was configured to start automatically.

6.1 Installation Steps

- Launched an EC2 instance and configured a security group permitting inbound access on port 8080 (Jenkins UI) and 22 (SSH).
- Installed Java (Jenkins runtime dependency), Git, Docker, AWS CLI, and kubectl.
- Installed Jenkins and enabled it as a system service so it restarts automatically with the instance.
- Added the jenkins system user to the docker group so pipeline builds can invoke Docker without elevated privileges.
- Unlocked Jenkins using the initial administrator password, installed the recommended plugin set, and created an administrator account.

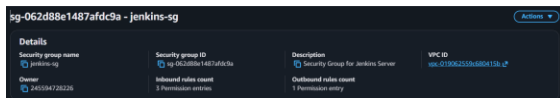


Figure 17: EC2 security group configured for Jenkins

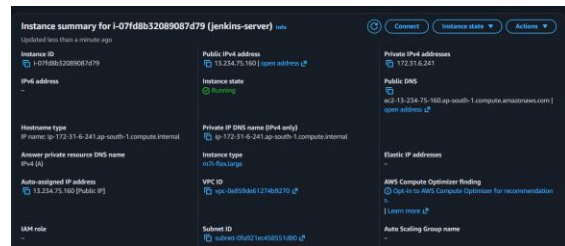


Figure 18: Jenkins EC2 instance in running state

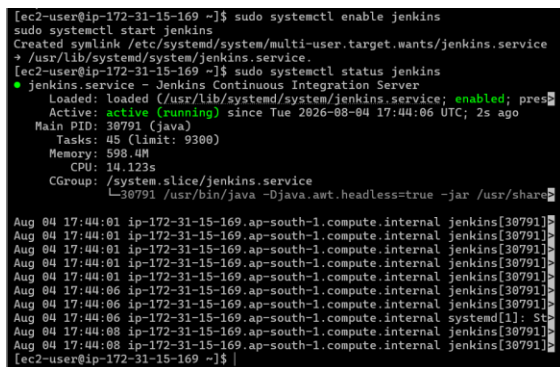


Figure 19: Jenkins service active and running

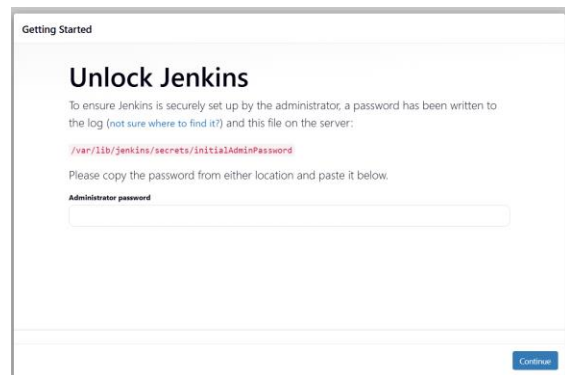


Figure 20: Jenkins initial unlock / setup screen

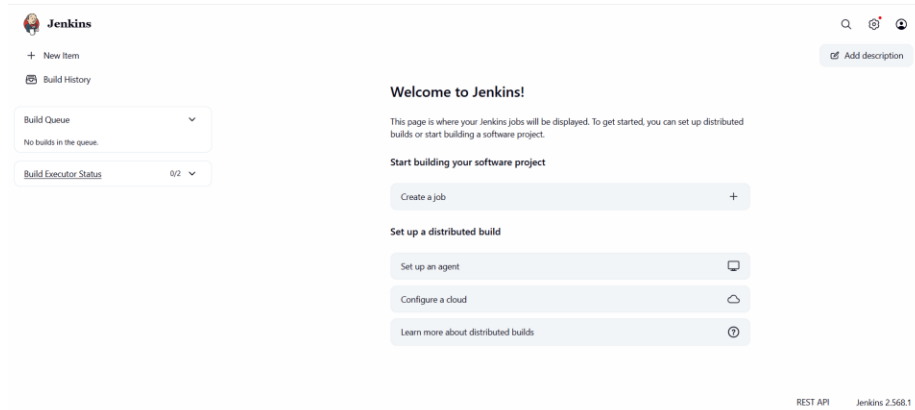


Figure 21: Jenkins dashboard accessible via browser

7. Amazon EKS Cluster Setup

The EKS cluster was provisioned with one control plane and a managed node group of at least two worker nodes, satisfying the project's high-availability requirement. Cluster connectivity was verified using kubectl from both a local machine and the Jenkins EC2 instance.

7.1 Cluster Provisioning

- Created the EKS control plane and a managed node group (≥ 2 worker nodes) using eksctl.
- Underlying infrastructure (VPC, subnets, security groups, IAM roles) provisioned automatically via CloudFormation stacks.
- Configured kubectl locally with: `aws eks update-kubeconfig --region <region> --name <cluster-name>`.
- Verified node readiness with `kubectl get nodes` and cluster reachability with `kubectl cluster-info`.

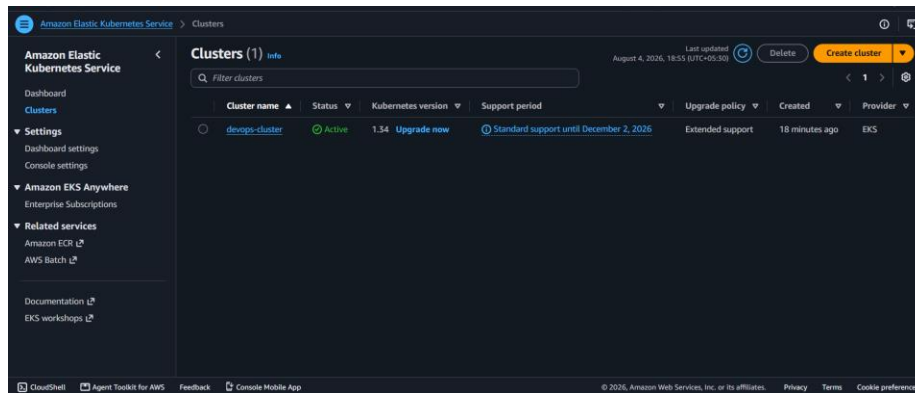


Figure 22: Amazon EKS cluster overview in the AWS Console

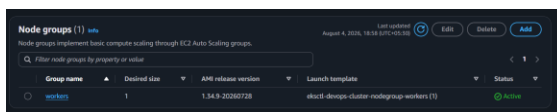


Figure 23: Managed node group — compute configuration

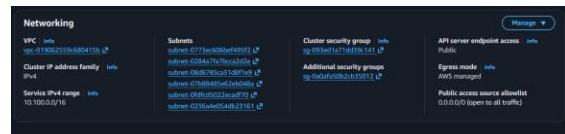


Figure 24: Cluster networking configuration

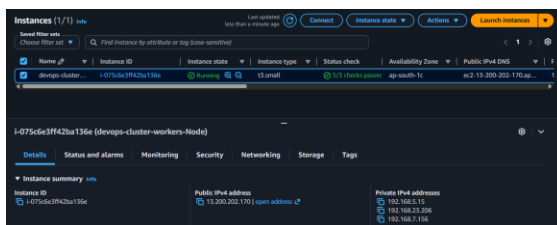


Figure 25: EC2 worker node backing the EKS cluster

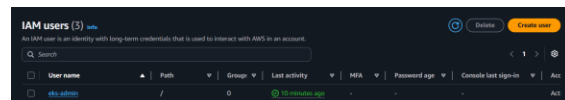


Figure 26: IAM roles associated with the EKS cluster

7.2 Cluster Verification



Figure 27: kubectl get nodes — worker nodes in Ready state



Figure 28: kubectl cluster-info — control plane reachable

```
pikun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app$ kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
aws-node-cfvnx                     2/2     Running   0           4m5s
coredns-677768ffb7-fqj8c           1/1     Running   0           7m27s
coredns-677768ffb7-m97bs           1/1     Running   0           7m27s
kube-proxy-7m66n                   1/1     Running   0           4m5s
metrics-server-6457d85649-jptvf    1/1     Running   0           2m45s
metrics-server-6457d85649-x5hbw    1/1     Running   0           2m45s
pikun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app$
```

Figure 29: Kube-system namespace pods confirming a healthy control plane

8. Jenkins Configuration for CI/CD

8.1 Global Tool Configuration

Under Manage Jenkins → Tools, Git was auto-detected and a NodeJS installation (matching the application's dependency requirements) was configured for use inside the pipeline via the tools directive.

8.2 Credentials

Two credentials were configured under Manage Jenkins → Credentials, allowing Jenkins to authenticate with GitHub and AWS without embedding secrets in the pipeline script itself.

Credential ID	Kind	Purpose
github-cred	Username with password (GitHub PAT)	Repository checkout
aws-cred	AWS Credentials	ECR authentication and EKS access

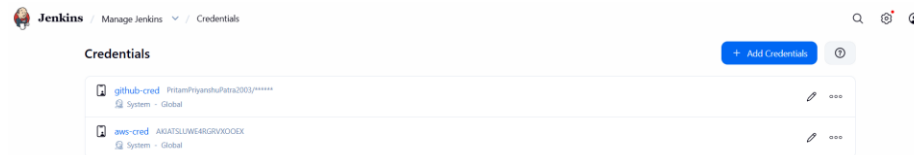


Figure 30: Configured Jenkins credentials — GitHub and AWS

8.3 EKS Connectivity from Jenkins

Before the pipeline could deploy to the cluster, connectivity from the Jenkins EC2 instance to EKS was verified, and the IAM identity used by Jenkins was mapped into the cluster's aws-auth ConfigMap — IAM permissions alone do not grant kubectl access; Kubernetes RBAC is a separate authorization layer.

```
PS C:\Users\Victus\Downloads> aws eks update-kubeconfig --region ap-south-1 --name devops-cluster
Added new context arn:aws:eks:ap-south-1:245594728226:cluster/devops-cluster to C:\Users\Victus\.kube\config
PS C:\Users\Victus\Downloads> kubectl get nodes
NAME                                     STATUS    ROLES    AGE      VERSION
ip-192-168-7-156.ap-south-1.compute.internal Ready    <none>   4h53m   v1.34.9-eks-bca9cf6
PS C:\Users\Victus\Downloads>
```

Figure 31: kubectl get nodes executed successfully from the Jenkins server

8.4 Pipeline Job Configuration

The pipeline job was configured as "Pipeline script from SCM", pointing directly at the Jenkinsfile stored in the application repository, so the pipeline definition is version-controlled alongside the application code.

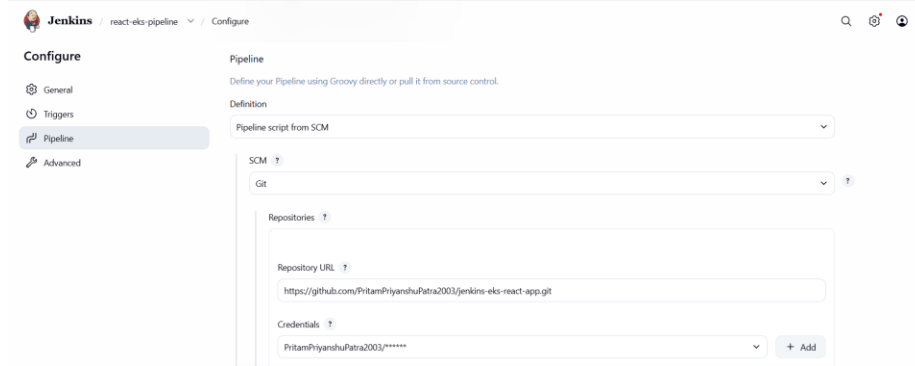


Figure 32: Jenkins pipeline job configured to pull the Jenkinsfile from GitHub

9. CI/CD Pipeline Workflow

The pipeline is defined declaratively in a Jenkinsfile stored at the root of the application repository. It executes seven stages in sequence, and any stage failure halts the pipeline before affecting the live deployment.

9.1 Pipeline Definition (Jenkinsfile)

```
pipeline {
  agent any
  tools { nodejs 'node20' }

  environment {
    ECR_REPO = '245594728226.dkr.ecr.ap-south-1.amazonaws.com/jenkins-eks-react-app'
    IMAGE_TAG = "${BUILD_NUMBER}"
    CLUSTER_NAME = 'devops-cluster'
    DEPLOYMENT_NAME = 'react-deployment'
    CONTAINER_NAME = 'react-container'
  }

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main',
            url: 'https://github.com/PritamPriyanshuPatra2003/jenkins-eks-react-app.git',
            credentialsId: 'github-cred'
      }
    }
    stage('Install Dependencies') { steps { sh 'npm install' } }
    stage('Build React App') { steps { sh 'npm run build' } }
    stage('Build Docker Image') {
      steps { sh "docker build -t $ECR_REPO:$IMAGE_TAG ." }
    }
    stage('Push to ECR') {
      steps {
        withCredentials([[ $class: 'AmazonWebServicesCredentialsBinding',
                           credentialsId: 'aws-cred' ]]) {
          sh """
            aws ecr get-login-password --region ap-south-1 | \\\
            docker login --username AWS --password-stdin $ECR_REPO
            docker push $ECR_REPO:$IMAGE_TAG
          """
        }
      }
    }
    stage('Update K8s Deployment') {
      steps {
        withCredentials([[ $class: 'AmazonWebServicesCredentialsBinding',
                           credentialsId: 'aws-cred' ]]) {
          sh """
            aws eks update-kubeconfig --region ap-south-1 --name $CLUSTER_NAME
            kubectl set image deployment/$DEPLOYMENT_NAME \\\

```



```
$CONTAINER_NAME=$ECR_REPO:$IMAGE_TAG  
kubectl rollout status deployment/$DEPLOYMENT_NAME  
"" ""  
}  
}  
}  
}  
}
```

9.2 Stage-by-Stage Summary

Stage	Purpose
Checkout	Clones the repository at the latest commit using stored GitHub credentials
Install Dependencies	Runs npm install to fetch project dependencies
Build React App	Produces the optimized production bundle with npm run build
Build Docker Image	Packages the build output into a Docker image tagged with the Jenkins build number
Push to ECR	Authenticates to Amazon ECR and pushes the newly built image
Update K8s Deployment	Updates the live Deployment's image and waits for the rollout to complete

9.3 First Pipeline Execution

The first run failed at the Install Dependencies stage due to a missing system library (libatomic) on the Jenkins host; after resolving it, the pipeline completed successfully end-to-end. Both runs are shown below as evidence of real debugging rather than an idealized first attempt.

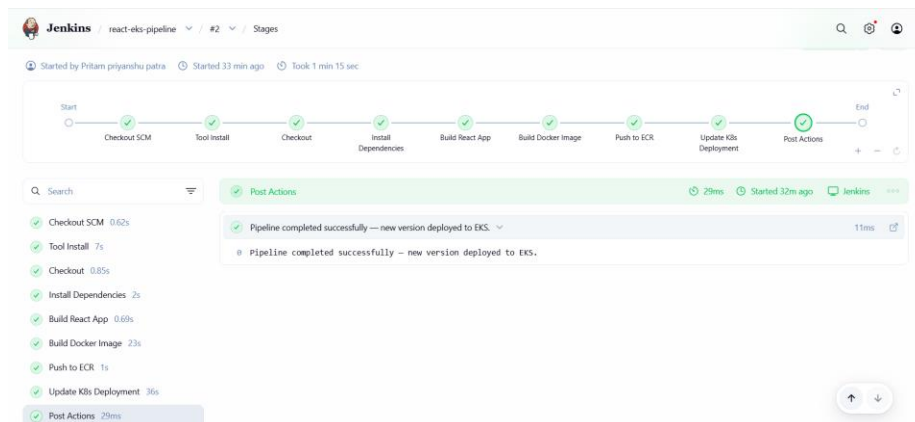


Figure 33: Pipeline stage view — build #1 failed at Install Dependencies, build #2 succeeded across all stages

```
Waiting for deployment "react-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "react-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "react-deployment" successfully rolled out
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully - new version deployed to EKS.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 34: Console output confirming a successful rollout and "Finished: SUCCESS"

10. Continuous Deployment

To satisfy the continuous-deployment requirement, Jenkins was configured to poll the GitHub repository every five minutes and automatically start a new pipeline run whenever a change is detected on the main branch — no manual "Build Now" action is required.

10.1 Poll SCM Configuration

Figure 35: Poll SCM trigger configured with schedule H/5 * * * * *

10.2 Verifying the Automatic Trigger

A small code change was committed and pushed to the repository. Within the polling interval, Jenkins detected the change and started a new build automatically, without any manual interaction — confirming genuine continuous deployment.

```
pikun@PIKUN: /mnt/d/Devops-projects/jenkins-eks-react-app/src$ git add .
pikun@PIKUN: /mnt/d/Devops-projects/jenkins-eks-react-app/src$ git commit -m
"Update UI text to test CD"
[main 1346822] Update UI text to test CD
1 file changed, 2 insertions(+), 2 deletions(-)
pikun@PIKUN: /mnt/d/Devops-projects/jenkins-eks-react-app/src$ git push origin
main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 424 bytes | 13.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/PritamPriyanshuPatra2003/jenkins-eks-react-app.git
e534369..1346822 main -> main
pikun@PIKUN: /mnt/d/Devops-projects/jenkins-eks-react-app/src$
```

Figure 36: Code change committed and pushed to GitHub



Figure 37: Jenkins automatically starting a new build

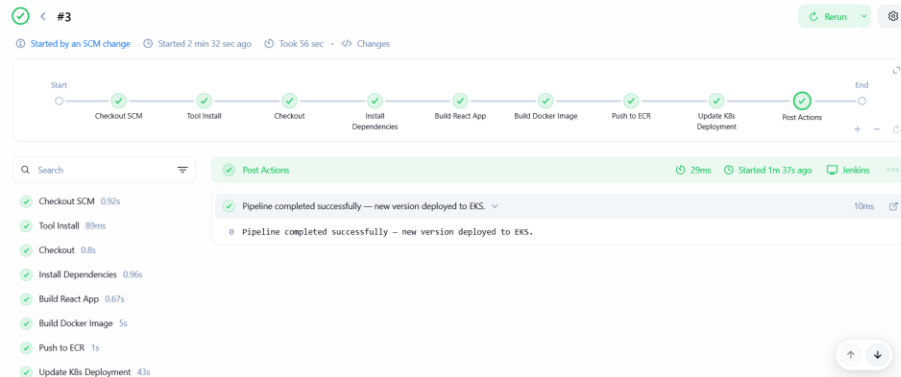


Figure 38: Automatically triggered build completing successfully

11. Kubernetes Deployment on Amazon EKS

The application is deployed to the EKS cluster using two manifests — a Deployment and a Service — applied via kubectl. These are the exact manifests used in the project repository's `k8s/` directory.

11.1 Deployment Manifest (`k8s/deployment.yaml`)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: react-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: react
  template:
    metadata:
      labels:
        app: react
    spec:
      containers:
        - name: react-container
          image: 245594728226.dkr.ecr.ap-south-1.amazonaws.com/jenkins-eks-react-app:2.1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
          envFrom:
            - configMapRef:
                name: react-config
```

11.2 Service Manifest (`k8s/service.yaml`)

```
apiVersion: v1
kind: Service
metadata:
  name: react-service
spec:
  selector:
    app: react
  ports:
    - port: 80
      targetPort: 80
  type: LoadBalancer
```

The Deployment references the `react-config` ConfigMap (Section 5.3) via `envFrom`, injecting application configuration as environment variables without rebuilding the image. The image tag (`2.1`) corresponds to the version deployed by the Jenkins pipeline at the time of writing; in production, this tag is updated automatically by the Update K8s Deployment stage on every successful build.

11.3 Cluster State Verification

```

ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/src$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
react-deployment-5f8c749574-5g8pj   1/1     Running   0           3m33s
react-deployment-5f8c749574-jzz8k   1/1     Running   0           3m36s
react-deployment-5f8c749574-x59p5   1/1     Running   0           3m34s
ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/src$

```

Figure 39: All application pods Running (3/3)

```

ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/src$ kubectl rollout
history deployment/react-deployment
deployment.apps/react-deployment
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         <none>
ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/src$

```

Figure 41: Rollout history across three successful revisions

```

ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/src$ kubectl get de
ployment
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
react-deployment                    3/3     3             3           5h41m
ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/src$

```

Figure 40: Deployment showing 3/3 ready and available replicas

```

ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/src$ kubectl get sv
c
NAME                                TYPE          CLUSTER-IP      EXTERNAL-IP      AGE
kubernetes                          ClusterIP      10.100.0.1       <none>           6h56m
react-service                       LoadBalancer  10.100.246.38    aa3e7ceca374c417cb7d2be8cb01c5b5-1557580701.ap-south-1.elb.amazonaws.com 80:31891/TCP 5h41m
ptkun@PIKUN:/mnt/d/Devops-projects/jenkins-eks-react-app/src$

```

Figure 42: Service exposed via AWS LoadBalancer, external DNS assigned

12. Application Live Verification

The final verification step confirms the entire pipeline delivers a working, publicly reachable application. The LoadBalancer's DNS name was opened in a browser, confirming the application — including a deliberate version marker used to prove the continuous-deployment trigger — is live and current.

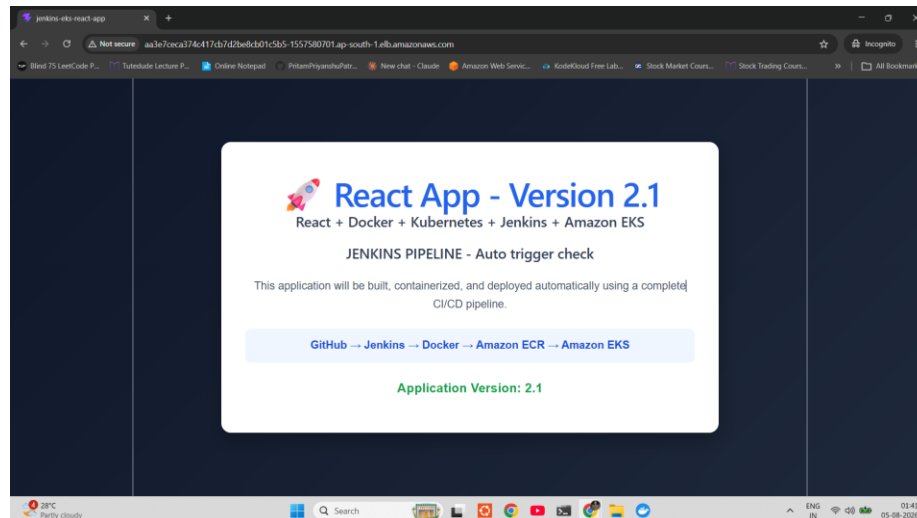


Figure 43: Application accessible in the browser via the EKS LoadBalancer URL

12.1 Final Build History

The complete build history shows one initial failed run (used for debugging) followed by consistently successful builds, including the automatically triggered run, demonstrating pipeline reliability across multiple executions.

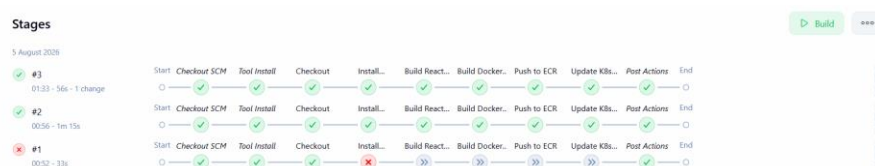


Figure 44: Jenkins build history — builds #1 through #3, including the auto-triggered run

13. Challenges Faced

Several real infrastructure issues were encountered and resolved during implementation. These are documented below as they reflect genuine troubleshooting rather than an idealized setup.

Issue	Root Cause	Resolution
Node.js failed with "error while loading shared libraries: libatomic.so.1"	Missing system library on a minimal Amazon Linux EC2 image	Installed the missing library with the system package manager (dnf/yum install libatomic)
Docker build stage failed with "permission denied" connecting to the Docker daemon	The jenkins system user was not a member of the docker group	Added the jenkins user to the docker group and restarted the Jenkins service
kubectl commands returned "Unauthorized" despite the IAM user having AdministratorAccess	IAM permissions and Kubernetes RBAC are separate authorization systems	Mapped the IAM user into the EKS cluster's aws-auth ConfigMap under mapUsers with the system:masters group

14. Learning Outcomes

- Gained hands-on experience provisioning and managing a production-grade Amazon EKS cluster, including managed node groups and cluster networking.
- Learned to host, configure, and secure a Jenkins server on EC2, including plugin management, global tool configuration, and credential handling.
- Understood the distinction between IAM permissions and Kubernetes RBAC, and how to bridge them via the aws-auth ConfigMap — a common real-world stumbling block.
- Built and debugged a declarative Jenkins pipeline covering source control, build, containerization, registry push, and Kubernetes rollout stages.
- Practiced core Kubernetes object management — Pods, Deployments, Services, ConfigMaps, Secrets, and Namespaces — and observed self-healing and rolling-update behaviour firsthand.
- Implemented and verified genuine continuous deployment, confirming that a Git push alone — with no manual steps — results in a new, live version of the application.
- Developed practical debugging skills for infrastructure-level issues (missing system libraries, Unix group permissions, cloud IAM-to-RBAC mapping) that are not covered by application code alone.

15. Conclusion

This project successfully demonstrates a complete, production-style CI/CD pipeline that takes a React application from a Git commit to a live, load-balanced deployment on Amazon EKS, with no manual steps at any stage. All ten project requirements — from cluster creation through continuous deployment — were implemented and verified with direct evidence, and the real infrastructure issues encountered along the way were diagnosed and resolved, reinforcing practical DevOps engineering skills across AWS, Jenkins, Docker, and Kubernetes.